Backpropagation

Lecture 4 — Turning a wrong network into a right one

Dr Thomas Robinson

August 6, 2026

LSE SUMMER SCHOOL · 2026

Part 1 · Loss & the goal

Where we are — and what we still need

We have

- A **forward pass**: take input $\mathbf{x}$, push it through the graph, get prediction $\hat{y}$.
- A **structure** — the computational graph — we can walk in either direction.
- A **local-derivative cheat-sheet** for every operation.

We don't have

- A way to **say how wrong** $\hat{y}$ is, compared to the true y.
- A way to **propagate that wrongness backwards** to update each parameter.

Today's job!

The loss — quantifying “wrongness”

For a prediction $\hat{y}$ and truth y, the **loss** $L(\hat{y}, y)$ is a function whose value is:

- **Zero** if the prediction is perfect.
- **Large and positive** if the prediction is far from the truth.
- **Differentiable** in $\hat{y}$.

For continuous outcomes (e.g. a regression):

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

i.e. the **squared error**.

For binary outcomes (e.g. classify spam vs not):

$$L(\hat{y}, y) = -\big(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})\big)$$

i.e. the **binary cross-entropy**.

(We’ll come back to think about categorical outcomes later!)

What those two losses actually punish

SAME FORMULAS AS THE LAST SLIDE — NOW READ THE NUMBERS

Squared error

Squared error chases the *big* misses first, and is easily dominated by outliers.

- E.g. being out by **4** is **sixteen** times worse than being out by 1.

CROSS-ENTROPY · TRUE LABEL $y = 1$

model says $\hat{y}$	$L = -\ln \hat{y}$	
0.9	0.11	confident, right
0.5	0.69	no idea
0.1	2.30	confident, **wrong**
0.01	4.61	very confident, very wrong

CROSS-ENTROPY PUNISHES CONFIDENT ERRORS HARDER THAN UNCERTAIN ONES

When $y = 1$, the penalty grows without limit as $\hat{y} \to 0$ (i.e. $0.01, 0.001, 0.00001$ etc.) – and vice versa when $y = 0$ and $\hat{y} \to 1$.

A model that "hedges its bets" when it doesn't know will score better than one that guesses boldly.

That is a *design choice*, baked into a formula and we can alter it, if we want.

What *is* a predicted probability?

ONE NUMBER, QUIETLY DOING TWO DIFFERENT JOBS

Your model says $\hat{y} = 0.7$ for one voter. Two readings — both standard, both defensible:

A claim about the world

"Among voters who look like this one, 70% turn out."

A **frequency**, in a reference class. The randomness is out there in the world.

Even a perfect model could not sharpen this.

Aleatoric — irreducible

A claim about the model

"I am 70% confident this voter turns out."

A **degree of belief**. This voter votes or doesn't — there is no 0.7 about *them*.

Better features or more data could sharpen it.

Epistemic — reducible

THE AWKWARD BIT

The loss cannot tell these apart. Cross-entropy just sees a number in $(0, 1)$; it is minimised, in expectation, by the *true conditional probability* $P(y = 1 \mid \mathbf{x})$.

So the maths is doing the first thing. We almost always *talk* as if it were the second.

Why the duality matters when you write up

$\hat{y} = 0.5$ means two opposite things

- *"Genuinely a coin flip."* → stop collecting data; you have hit the ceiling.
- *"I have no idea."* → collect more data; the ceiling is higher than this.

The number alone will not tell you which.

$\hat{y}$ COMES WITH NO ERROR BAR

0.7 from a model trained on 50 cases and 0.7 from one trained on 50 million look **identical**.

The output is a point estimate of a probability — not a probability *distribution* over that estimate.

WHAT YOU *CAN* CHECK

Calibration. Take every case where the model said ≈0.7. Did about 70% of them turn out?

That tests the *frequency* reading, across a group. It says nothing about whether 0.7 was the right belief for any **one** voter — and that is usually the claim a reader cares about.

⚠ Be precise in your write-up about which reading you mean. "The model is 70% confident" and "70% of such voters turn out" are different sentences, and only one of them is what you fitted.

A worked turnout example, by the numbers

BACK TO YESTERDAY'S TURNOUT PROBLEM

Suppose one voter has $x = 1$, true outcome $y = 2$ (a fictional "turnout score"). Our (very wrong) model is a single **linear** neuron with $w = 0$, $b = 0$.

Forward pass:

- $z = wx + b = (0 \cdot 1) + 0 = 0.$
- Loss: $L = (z - y)^2 = (0 - 2)^2 = 4.$

(Here z is our prediction $\hat{y}$ — for a single linear neuron they're the same thing.)

QUESTION

Should we increase w or decrease w to reduce the loss?

ANSWER

Increase w. If w were 2 (true relationship), our prediction would be $z = 2$, and loss would be 0.

So the question becomes: how can an *algorithm* know that?

Why not just try every value?

THE HONEST QUESTION FROM THE LAST SLIDE

Guess and check

Try 100 values of w, keep the best.

- 1 parameter → 100 forward passes.
- 2 parameters → 100² = 10,000.
- 10 parameters → 100^{10}.

Hopeless past a handful of parameters.

Numerical differentiation

Nudge each parameter and see what happens:

$$\frac{\partial L}{\partial w} \approx \frac{L(w+h) - L(w)}{h}$$

- Needs **one forward pass per parameter**.
- Approximate — and sensitive to the choice of h.

WHAT WE WANT

For a model with k parameters:

- Minimum number of passes (i.e. not k)
- An exact measure of the change in loss for a change in the parameter

It turns out, even with $k > 1\ \text{million}$, we can do this with just *two* passes!

What we want to compute

THE GRADIENT OF THE LOSS WITH RESPECT TO EACH PARAMETER

For each weight w and bias b, we want:

$$\frac{\partial L}{\partial w}, \qquad \frac{\partial L}{\partial b}$$

A negative $\frac{\partial L}{\partial w}$ means: "increase w to reduce L."

A positive $\frac{\partial L}{\partial w}$ means: "decrease w to reduce L."

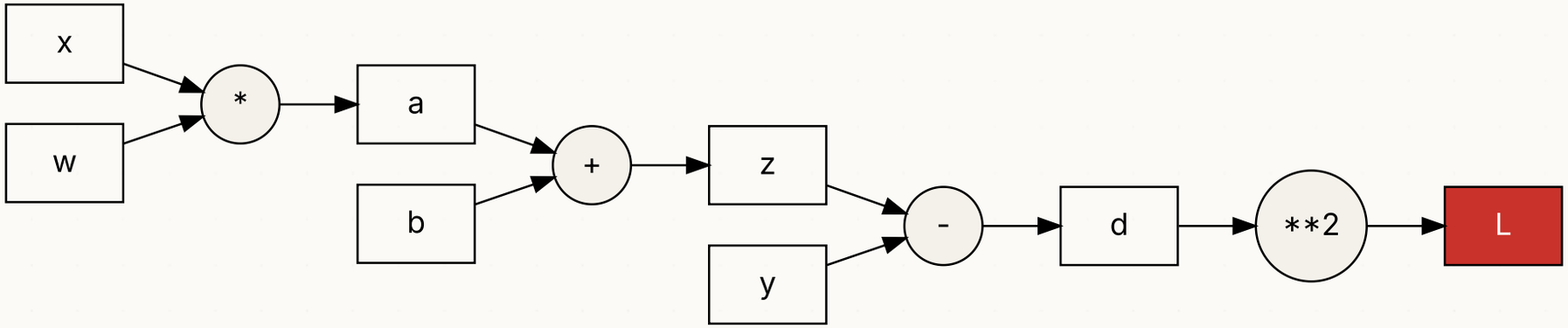
The **size** of the partial tells us how strongly the loss responds — i.e. how big a step we can take.

Computing this from the *full expression* of L is painful, but...

- Using the chain rule, applied node-by-node, we can make it mechanically easy to compute

Adding loss to the graph

Take our linear perceptron $z = wx + b$, and append loss nodes for $L = (z - y)^2$:



NAME THE NODES

Reading the graph left to right, each circle is one operation with its own output:

- $a = wx$ — multiply the weight by the input
- $z = a + b$ — add the bias
- $d = z - y$ — subtract the truth
- $L = d^2$ — square the error

The loss is just another node in the graph. Now there's a single scalar at the end — the perfect target for reverse-mode autodiff.

Backwards through the graph — one step at a time

We want $\frac{\partial L}{\partial w}$.

Walking the graph **right to left**, we apply the chain rule one node at a time.

Starting at L: $\frac{\partial L}{\partial L} = 1$ (every node's gradient w.r.t. itself is 1).

The squared step. $L = d^2$ so $\frac{\partial L}{\partial d} = 2d$.

We got this using only the *local derivative* of `**2` — independent of everything else.

We can continue this back through each node using only the local derivative at each step

- The chain rule *accumulates* the result

The full backward walk

WORKING BACKWARDS THROUGH THE LINEAR PERCEPTRON + SQUARED-ERROR LOSS

$$
\begin{aligned}
\frac{\partial L}{\partial L} &= 1 \\
\frac{\partial L}{\partial d} &= 2d && (\text{derivative of } d^2) \\
\frac{\partial L}{\partial z} &= \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial z} = \frac{\partial L}{\partial d} \cdot 1 = 2d && (d = z - y) \\
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial a} = \frac{\partial L}{\partial z} \cdot 1 = 2d && (z = a + b) \\
\frac{\partial L}{\partial w} &= \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial w} = \frac{\partial L}{\partial a} \cdot x = 2d \cdot x && (a = w \cdot x) \\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b} = \frac{\partial L}{\partial z} \cdot 1 = 2d
\end{aligned}
$$

Tip

Every step uses only: the local operation's derivative + the gradient flowing in from the right. That's why this scales — each node only "talks" to its neighbours.

With concrete numbers

Plug in our turnout example: $x = 1, y = 2, w = b = 0$.

Forward: $a = 0, z = 0, d = -2, L = 4$.

BACKWARD

$$\frac{\partial L}{\partial w} = 2d \cdot x = 2(-2)(1) = -4$$

$$\frac{\partial L}{\partial b} = 2d = -4$$

THE SIGNAL

$\frac{\partial L}{\partial w} = -4 < 0$ — so increasing w will *decrease* the loss. Just what we hoped.

The algorithm found this **without** trying every value 😎

Stop and check — what about an activation function?

STOP & CHECK

TRY IT YOURSELF (LOOK AT THE DERIVATIVES CHEATSHEET!)

Suppose we wrap the linear output in a **sigmoid** activation: $z = \sigma(wx + b)$ where $\sigma(u) = 1/(1 + e^{-u})$.

What single thing changes in the backward walk?

ANSWER

Exactly one local derivative changes. At the σ node:

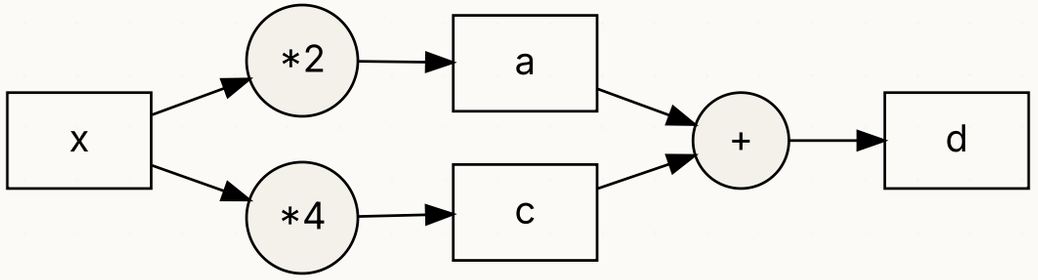
$$\frac{\partial z}{\partial(\text{pre-activation})} = \sigma'(u) = \sigma(u)(1 - \sigma(u))$$

— a value between 0 and 0.25. That extra factor gets multiplied into the chain wherever the gradient flows back through the activation.

Everything else is unchanged. **The whole algorithm is robust to swapping in any operation we can differentiate.**

Branches: when a value flows into more than one operation

In real networks, the same value gets used in multiple downstream operations.



Branching rule. When a value x goes to *multiple* downstream nodes, its gradient is the **sum** of contributions from each branch:

$$\frac{\partial d}{\partial x} = \underbrace{\frac{\partial d}{\partial a} \cdot \frac{\partial a}{\partial x}}_{\text{top branch}} + \underbrace{\frac{\partial d}{\partial c} \cdot \frac{\partial c}{\partial x}}_{\text{bottom branch}}$$

This is the only extra rule beyond the chain rule. Everything else is bookkeeping.

WITH NUMBERS

Here the branches are $a = 2x$ and $c = 4x$, and $d = a + c$. So the local derivatives are $\frac{\partial a}{\partial x} = 2$, $\frac{\partial c}{\partial x} = 4$, and $\frac{\partial d}{\partial a} = \frac{\partial d}{\partial c} = 1$. Summing the two branches:

$$\frac{\partial d}{\partial x} = \underbrace{1 \cdot 2}_{\text{top}} + \underbrace{1 \cdot 4}_{\text{bottom}} = 6$$

Backpropagation

SENDING LOSS GRADIENTS FROM RIGHT TO LEFT

We walk backwards through a graph, applying the chain rule at every node, and reading off $\frac{\partial L}{\partial w}$ — without ever expanding the full expression of L.

This is called reverse-mode automatic differentiation — or, more normally, *backpropagation*.

It is the **single algorithm** that powers every modern neural network. It was published by Rumelhart, Hinton & Williams in 1986 (Rumelhart et al., 1986).

- Scales linearly with the size of the graph (i.e. number of parameters)
- Works for arbitrary architectures (including vision models, LLMs, etc.)
- Is *not* approximate — the gradients are exact.

We will use backpropagation every single day for the rest of the course.

:::

Plenary · backward pass by hand

Plenary · backward pass by hand

≈ 20 MIN · PEN AND PAPER, IN PAIRS

THE GRAPH

$$y = (3x + 2)(x - 1) \quad \text{with } x = 4.$$

Break it into operations:

- $a = 3x$
- $b = a + 2$
- $c = x - 1$
- $y = b \cdot c$

COMPUTE, STEP BY STEP:

1. Forward: values of a, b, c, y.
2. Backward: $\frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}, \frac{\partial y}{\partial a}, \frac{\partial y}{\partial x}$.
3. Sanity check: differentiate $y = (3x + 2)(x - 1)$ algebraically and evaluate at $x = 4$. Do you get the same number?

Plenary · answers

FORWARD

$a = 12,\ b = 14,\ c = 3,\ y = 42.$

BACKWARD

$\frac{\partial y}{\partial y} = 1.$

$\frac{\partial y}{\partial b} = c = 3$ (since $y = b \cdot c$).

$\frac{\partial y}{\partial c} = b = 14.$

$\frac{\partial y}{\partial a} = \frac{\partial y}{\partial b} \cdot 1 = 3.$

x **flows into two branches** — through a and through c:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial a} \cdot 3 + \frac{\partial y}{\partial c} \cdot 1 = 3 \cdot 3 + 14 \cdot 1 = 23.$$

SANITY CHECK

$y = (3x + 2)(x - 1) = 3x^2 - x - 2.\ \frac{dy}{dx} = 6x - 1 = 23$ at $x = 4.$ ✅

Part 2 · The algorithm in full

Three steps, repeated

For any parametric model, training is:

1. Measure

Compute the **loss** on a training example (or batch of examples).

- Forward pass through the graph.

2. DIFFERENTIATE

Compute ∇L — the gradient of the loss w.r.t. every parameter.

- Backward pass through the graph.

3. Update

Step each parameter in the direction that **reduces** L.

- Gradient descent.

Repeat thousands of times. That's deep learning.

Reminder: forward and backward

Forward pass

Walk graph **left → right**.

For each node:

- Compute its output from its inputs.
- **Store** the output (we need it for backward).

End: a scalar loss.

BACKWARD PASS

Walk graph **right → left**.

For each node:

- Read the gradient flowing in from the right.
- Multiply by the local derivative (chain rule).
- Send the result to upstream nodes.

End: ∇L for every parameter.

Tip

Notice that the forward pass stores values; the backward pass reads them. This memory cost is why GPUs need a lot of RAM during training.

The update — gradient descent

For each parameter θ, do:

$$\theta \leftarrow \theta - \lambda \frac{\partial L}{\partial \theta}$$

- λ — the **learning rate** — controls step size.
- Negative gradient direction → loss decreases.
- Repeat for every parameter.

BACK TO OUR TURNOUT EXAMPLE

$w = 0, b = 0, x = 1, y = 2.$

After the backward pass, we had $\frac{\partial L}{\partial w} = -4$ and $\frac{\partial L}{\partial b} = -4$. With $\lambda = 0.1$, **what are w and b after one step?**

ANSWER

$w_{\text{new}} = 0 - 0.1 \cdot (-4) = 0.4.$ $b_{\text{new}} = 0 - 0.1 \cdot (-4) = 0.4.$

Forward pass with new parameters: $z = 0.4 + 0.4 = 0.8 \Rightarrow L = (0.8 - 2)^2 = 1.44.$

The loss fell from 4 to 1.44. *In one step.*

Several steps of gradient descent

SAME EXAMPLE, REPEATED

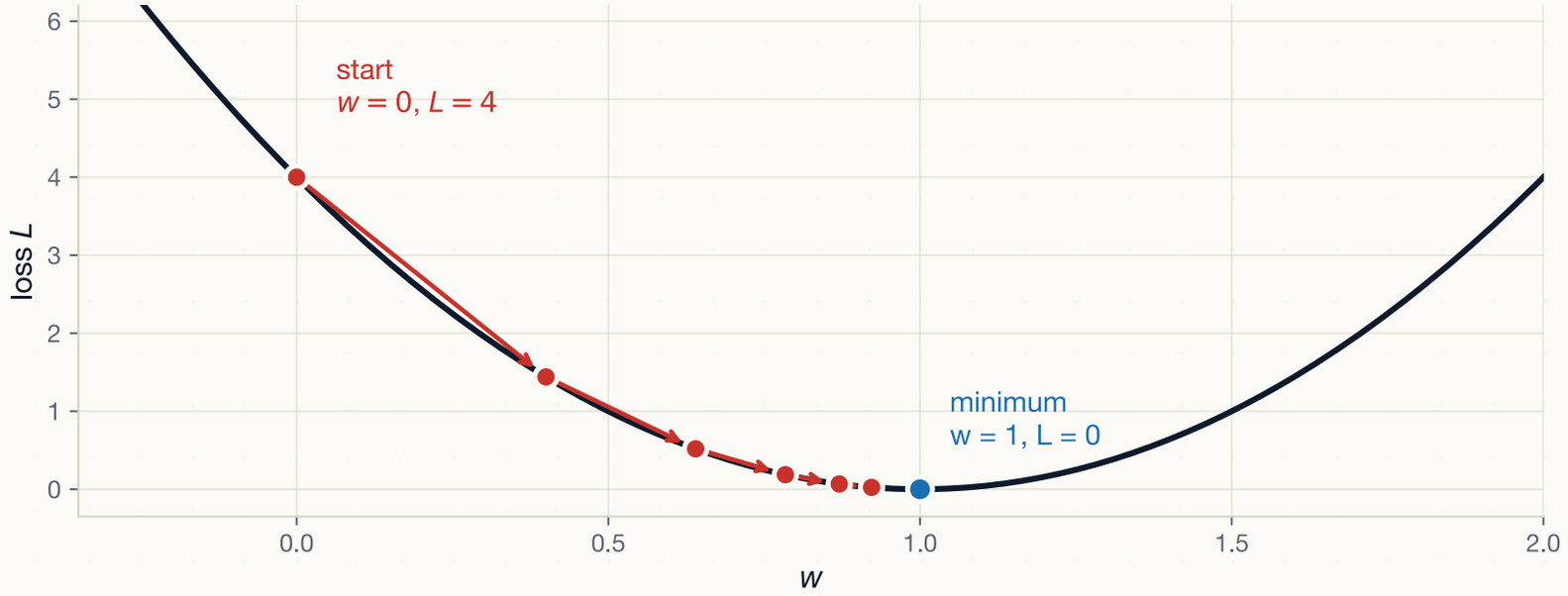
step	w	b	$z = wx + b$	L
0	0.00	0.00	0.00	4.00
1	0.40	0.40	0.80	1.44
2	0.64	0.64	1.28	0.52
3	0.78	0.78	1.57	0.19
4	0.87	0.87	1.74	0.07
5	0.92	0.92	1.84	0.02

The loss falls after each step, simply repeating the same forward/backward/update sequence.

Notice that the loss reduction decreases per epoch – suggesting we are approaching some limit (in this case we should hit zero exactly!)

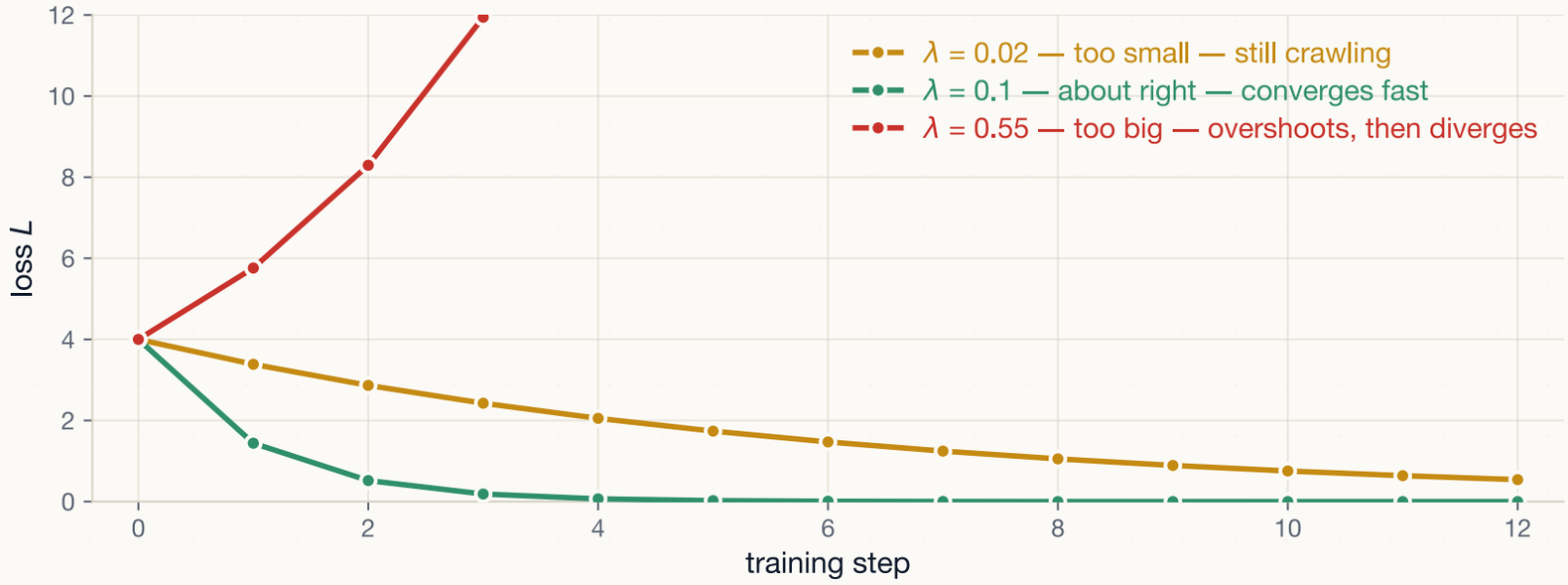
The same descent, as a picture

w AND b START EQUAL AND GET THE SAME GRADIENT, SO THEY MOVE TOGETHER — ONE AXIS SHOWS BOTH



Each arrow is one **forward → backward → update** cycle. The gradient tells us which way is downhill *and* how steep it is, so the steps are large where the curve is steep and shrink as we approach the bottom.

The learning rate — the one toggle you always tune



⚠ The red line leaves the top of the chart and never comes back. **A diverging loss is almost always the learning rate.** It's the first thing to halve when training blows up.

Does gradient descent always work?

Our example was easy

A single linear neuron with squared error gives a **convex** loss – it's "bowl-shaped" with a single minima.

- Hence, if you move downhill from anywhere and you reach *the* minimum.

Real networks are not

By stacking layers and using non-linear activations, the surface becomes **non-convex**

- There could be many *local* minima
- Lots of *saddle points* (flat in some directions, sloped in others).

There is no guarantee you find the global minimum. In general, you don't.

WHY WE DO IT ANYWAY

Empirically, for large networks, descending to local minima tends to be **good enough**.

- But this is an empirical observation, not a mathematical theorem.

Deep learning is therefore **experimental**: it works far better than anyone could prove it should!

A (slightly) more realistic example

Suppose we have two observations — $(x = 1, y = 2)$ and $(x = 3, y = 6)$ — what should we do?

STOCHASTIC GRADIENT DESCENT (SGD)

Update parameters using **one example at a time**.

- Cheap per step.
- Updates are **noisy** — but the noise averages out.
- For our two-example case: alternate between the two; the model converges to $w = 2, b = 0$.

Full-batch gradient descent

Compute the gradient by **averaging** over all training examples, then take one step.

- Smoother
- Memory-hungry for large datasets.
- Only **one slow step per epoch** — slower to converge.

Mini-batch gradient descent — the modern default

Compute the gradient on a **small batch** (say, 32 examples), then average, then make an update.

- Less noisy than single-example SGD.
- Memory-feasible even on huge datasets.
- Plays well with GPU parallelism (one batch = one matmul).

This is what every modern training script does.

Note

Vocabulary check. “SGD” in modern practice usually means mini-batch SGD — even though strictly speaking the original SGD used one example at a time.

Some essential training vocabulary

Training set
: Data used to compute updates.

Validation set
: Data used to **tune** hyperparameters (e.g. λ) and detect overfitting.

Test set
: Used **once**, at the very end, to estimate generalisation.

Step (or iteration)
: One parameter update.

Epoch
: One complete pass over the training set. *With mini-batches of size 32 and 10,000 examples, an epoch is ~313 steps.*

Convergence
: Loss stops decreasing meaningfully.

⚠ **A common bug**

As with "regular" ML, we must not *leak* validation/test data into training. If you do, the model **looks** great, but collapses out of sample.

Activity · one step of gradient descent

Activity · one step of gradient descent

≈ 15 MIN · PEN AND PAPER, IN PAIRS

THE MODEL AND ONE OBSERVATION

A linear perceptron:

$$z = wx + b, \quad L = (z - y)^2.$$

Current parameters: $w = 0.5, b = 0.1$. Example: $x = 2, y = 3$. Learning rate $\lambda = 0.1$.

COMPUTE

1. The forward pass: both z, and then L.
2. Backward pass: $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$.
3. Update new w and new b values.
4. **Forward pass with these new parameters** to confirm the loss has fallen.

Activity · answer

FORWARD

$z = 0.5 \times 2 + 0.1 = 1.1.\ d = z - y = 1.1 - 3 = -1.9.\ L = (-1.9)^2 = 3.61.$

BACKWARD

$\frac{\partial L}{\partial w} = 2d \cdot x = 2(-1.9)(2) = -7.6.\ \frac{\partial L}{\partial b} = 2d = -3.8.$

UPDATE WITH $\lambda = 0.1$

$w \leftarrow 0.5 - 0.1(-7.6) = 1.26.\ b \leftarrow 0.1 - 0.1(-3.8) = 0.48.$

RE-FORWARD

$z = 1.26(2) + 0.48 = 3.0 \rightarrow L = (3.0 - 3)^2 = 0.$

We hit the target *in one step* — because the loss is quadratic and we happened to land exactly. In real problems, expect dozens to thousands of steps.

Part 3 · Optimisers and the loop

Plain SGD has limits

Problems

- Fixed learning rate λ — too big, you overshoot; too small, you crawl.
- All parameters get the same step size — even when some need to move faster.
- Noisy steps can oscillate near minima.

The modern fix

Adaptive optimisers adjust λ per parameter — most often using **Adam** (Kingma, 2014) and its sibling **AdamW**.

We'll use these in every PyTorch model from Day 5.

- They are a good default for most DL projects

Adam, in one slide

FOR EACH PARAMETER θ, ON EACH STEP:

1. Compute the gradient $g_t = \nabla_\theta L$.
2. Maintain two moving averages:
   - $m_t \approx$ recent average of g (the **momentum**).
   - $v_t \approx$ recent average of g^2 (the **second moment**).
3. Update:

$$\theta \leftarrow \theta - \lambda \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Note

The point isn't the formula — it's that the update is **adaptive per parameter**. Parameters with large recent gradients get smaller scaled steps; parameters with small gradients get relatively larger ones.

In practice, **default to Adam with** $\lambda = 10^{-3}$ for most projects. We'll do exactly that next week.

The training loop — preview of next week

```
for epoch in range(num_epochs):
    for x_batch, y_batch in train_loader:
        optimizer.zero_grad()               # clear last step's gradients
        y_pred = model(x_batch)             # forward pass
        loss   = loss_fn(y_pred, y_batch)   # measure
        loss.backward()                     # backward pass — fills .grad on parameters
        optimizer.step()                    # update parameters
```

Every PyTorch training script you'll ever read or write looks like this. **Five lines.** Everything else is detail on:

- What `model` is (the architecture).
- What `loss_fn` is.
- What `optimizer` is.

We spend the next eight lectures filling in those three blanks.

A practical reminder — zero the gradients

Critical detail: PyTorch *accumulates* gradients across `loss.backward()` calls by default.

You must call `optimizer.zero_grad()` at the start of every step, or your gradients will be the sum of all previous steps' gradients!!

This is a very common bug in PyTorch code.

⚠ **Enormous gradients are a tell-tale sign**

Check whether you're zeroing them. It is almost always this.

Where this leaves us

STOP & TAKE STOCK

You can now:

1. **Define a loss** — squared error or binary cross-entropy.
2. Run a **forward pass** through a small computational graph.
3. Run a **backward pass** and read off gradients for every parameter.
4. Take a **gradient descent step** and confirm the loss falls.
5. Name the three components of any training loop: model, loss, optimiser.

You understand the algorithm that runs every neural network.

Three weeks ago this would have been a black box. Today you can write it.

A connection to social science

AN ASIDE ON A PIECE OF WORK YOU CAN READ

MIDAS (Lall & Robinson, 2022) is a missing-data imputation method built on a *denoising autoencoder* trained with backprop and Adam.

- It's used in political science / development economics for surveys with missing entries.
- The core algorithm — forward pass, loss, backward pass, update — is the same as today's.

The deep-learning idea you just learned is **applicable** to social-science problems, not just to ImageNet. We come back to MIDAS on Day 7.

Lab brief · today’s afternoon

Today’s lab — 90 minutes

GOALS

Take yesterday’s `Value` graph library and add **gradients**.

You will implement:

1. A `grad` attribute on each `Value`, initialised to 0.
2. For each operation (`+`, `*`, `**`), a `_backward` hook that:
   - Reads the gradient flowing in.
   - Computes the local derivative.
   - Adds the contribution to each child’s `grad`.
3. A `backward()` method on the final node that walks the graph in **reverse topological order**, calling each `_backward`.

Deliverable. A working autodifferentiation library, in <100 lines of Python.

You’ll then use it to train a tiny one-layer model on a small dataset — and watch the loss fall.

This is **Karpathy’s micrograd**. We’ve walked it deliberately.

See you next week

LECTURE 5 — *FEED-FORWARD NETWORKS IN PYTORCH* — MONDAY 10 AUGUST

Over the weekend:

- Well done! Week 1 is the steepest part of the whole course
- If you're curious, watch Karpathy's *Building micrograd* – it walks through the exact lab you just did.
- Optional: install PyTorch locally if you want to run things outside Colab. Setup notes are on Moodle.

Kingma, D. P. (2014). Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*.

Lall, R., & Robinson, T. (2022). The MIDAS touch: Accurate and scalable missing-data imputation with deep learning. *Political Analysis*, *30*(2), 179–196.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536.